

六槽语义框架：面向 Agent 通信

—— 定义 RSF (递归语义框架)

版本: v1.0 | 日期: 2026-07-14 | 状态: 概念框架 + 初步案例验证 | 协议: CC BY 4.0

摘要

RSF (Recursive Semantic Framework, 递归语义框架) 是一种面向智能系统的结构化语义中间表示协议。其核心是固定六槽骨架——A (施事)、B (动作)、C (受事)、D (时间)、E (地点)、F (方式/背景)——用于从自然语言中提取事件语义，并在智能系统之间直接交换语义结构，而非依赖自然语言作为中间媒介。

RSF 不是一种新语言，不是传统语法体系的替代品，也不是 LLM 或 Transformer 的竞争方案。它是一层位于自然语言与智能系统之间的语义接口。

RSF 围绕三个核心特性构建其价值：**一致性**（所有 Agent 基于同一语义结构工作，消除各自解析 NL 带来的差异）、**可追溯性**（每个输出片段可回映到原始输入的特定槽位，语义路径可复查）、**可审计性**（结构化的六槽日志支持逐项检查决策依据，适用于金融、法律、医疗等高可靠性场景）。

本文档记录了 RSF 的核心思想、结构定义、设计原则以及应用示例，包括多 Agent 链式通信、高保真领域翻译和 RAG 结构化等场景。当前 RSF 处于概念框架与初步案例验证阶段，尚无自动化解析器或大规模系统部署。本文档的目的是建立思想原创的时间记录，为后续研究提供公开参考。

1. 引言：交流的本质是意义而非语言

1.1 便利店：没有语言的交流

一名外国游客走进一家便利店。游客不会当地语言，店员也不会游客的语言。双方没有任何共同语言。游客想购买两瓶可乐——他没有说话，也没有写字，而是走到冰柜前，取出两瓶可乐，放在柜台上。店员观察这一行为后，举起两个手指。游客理解这一手势代表交易中的数量或价格关系，支付了两美元。店员收款。交易完成。

整个过程没有语言交换，没有文字交换，没有翻译——但交流成功发生了。

为什么？因为双方共享了一个具有约束作用的语义环境。便利店本身提供了关键的语义约束：游客拿起可乐这一行为，在仓库中可能表示搬运，在朋友家可能表示取用，但在便利店中，高度可能表示购买。

环境限制了行为的解释空间。同样，店员举起两个手指——脱离环境，它可能代表数字 2、第二名或胜利手势——但在交易场景中，其意义被当前环境和双方的共同知识所约束。

因此，交流并不依赖于符号之间的一一对应。它依赖于：符号 + 环境约束 + 共同知识 + 语义理解。

1.2 语言与意义分属不同层次

便利店案例揭示了一个重要事实：交流的核心不是语言本身，而是语言背后的意义结构。语言只是语义的一种表达形式，而不是语义本身。

这个区分看似简单，却指向一个被长期忽视的问题：当人工智能系统之间交换信息时，它们是否必须像人类一样依赖自然语言？还是可以直接交换意义本身？

不同语言可以表达相同的底层语义：

英文: The company completed the project.
中文: 公司完成了项目。

两个句子表面形式不同，但它们描述的是同一个事件：某个主体执行了一个行为，作用于某个对象。语言具有多样性，但语义结构具有更高层的一致性。

1.3 语义漂移：多 Agent 系统中的隐形成本

当前多 Agent 系统的通信仍然大量依赖自然语言。典型的过程是：

Agent A → 生成自然语言 → Agent B → 重新解析自然语言 → 提取意义 → 继续处理

这种方式直接沿袭了人类的交流模式。它具有灵活性，但也带来一个根本性问题：每一次通信，都需要重新从语言形式中恢复意义。

更严重的是，存在一个已知但尚未被充分解决的风险——语义漂移（Semantic Drift）。当 Agent A 将内部语义转换为自然语言时，信息会发生第一次衰减（编码损失）；当 Agent B 从自然语言中重建语义时，会发生第二次衰减（解码损失）。在长链条的多 Agent 协作中，每经过一次 NL 生成→NL 解析的循环，语义就发生一次不可控的变化。

考虑一个场景：Agent A（客服）通知 Agent B（仓库）“客户退回一个有缺陷的笔记本电脑”。Agent B 理解后通知 Agent C（财务）“给这个退货客户退款”。Agent C 收到的信息中，“有缺陷”可能已经退化为“退货”，“退款条件”可能丢失，退款金额是否包含运费可能完全没有提及。每经过一个 Agent，信息就发生一次不可追踪的变化。

这不是自然语言的“表达能力不足”问题——自然语言在表达复杂语义方面极为强大。问题在于，自然语言是为人类通信优化的，而不是为机器间的精确、高效、可验证的信息交换优化的。

1.4 RSF 的动机

基于上述观察，RSF（Recursive Semantic Framework，递归语义框架）尝试回答一个问题：如何让智能系统更加直接地表示和交换意义，而不是每次都依赖自然语言作为中间媒介？

RSF 不试图替代自然语言——自然语言仍然是人机交互的重要方式。它关注的是在智能系统内部，信息交换是否可以基于语义结构而非语言形式。

接下来的章节将定义 RSF 是什么、如何工作，以及它在多 Agent 通信和高保真领域翻译等场景中的初步应用。

2. RSF 的定义与核心结构

2.1 RSF 是什么

RSF（Recursive Semantic Framework）是一种面向智能系统的结构化语义中间表示协议。核心目标是从自然语言表达中提取事件语义，并以固定的六槽结构进行表示，使不同智能系统能够在语义层面直接交换信息，而不需要每次都重新解析自然语言。

RSF 不是：

- 一种新的自然语言或人工语言
- 传统语法体系的替代品
- LLM 或 Transformer 的竞争方案
- 通信协议（如 MCP、A2A）

RSF 是：一层位于自然语言与智能系统之间的语义接口。

2.2 六槽定义

RSF 将一个基础语义事件表示为六个槽位：

事件 = A(施事) + B(动作) + C(受事) + D(时间) + E(地点) + F(方式/背景)

槽位	名称	说明
A	施事 (Agent)	动作的执行主体
B	动作 (Action)	动词及其整体变形
C	受事 (Target)	动作作用的对象
D	时间 (Time)	时间点、时段等

E	地点 (Environment)	空间、方位等
F	方式/背景 (Context)	方式、原因等统一入 F

例如：

The company completed the project yesterday in Tokyo.

A	B	C	D	E	F
Company	completed	project	yesterday	Tokyo	∅

RSF 关注的不是“句子由哪些词组成”，而是“这个表达描述了什么事件结构”。同一个事件，无论用英语、中文还是日语表达，经过 RSF 提取后应当产生一致的六槽骨架。

2.3 递归：六槽套六槽，不增新槽名

RSF 名称中的“Recursive”（递归）代表了其核心设计思想之一：复杂性来自结构的递归展开，而不是规则数量的增加。

递归的定义是：当六槽中的任何一个槽位内部包含一个完整事件时，可以用同一套 A - F 框架进入下一层，不增设新槽名。

例如：

The researchers who conducted the study analyzed the data.

第一层：

A	B	C
The researchers [who conducted the study]	analyzed	the data

递归进入 A 槽（定语从句）：

A	B	C
who（指 researchers）	conducted	the study

递归层仍然使用 A、B、C，而不是 A1、B1、C1。无论嵌套深度是 1 层还是 10 层，解析规则完全相同。Core 不需要为深度嵌套准备任何额外规则。

2.4 输出模式

RSF 提供三种输出模式：

模式	内容	适用场景
轻量模式	仅最终结果（译文或响应）	日常使用
标准模式	结果 + 符号标记	需要基本可追溯性
审计模式	结果 + 树状 JSON + 调整记录	高可靠性场景

2.5 与现有通信协议的关系

MCP（Model Context Protocol）和 A2A（Agent-to-Agent）解决的是 Agent 之间如何连接、如何发现工具、如何传输数据的问题。RSF 与它们不构成竞争关系：

层次	职责	示例
通信协议层	Agent 如何连接和数据如何传输	API / MCP / A2A
语义表示层（RSF）	Agent 交换的意义是什么	六槽结构化语义
应用层	具体的业务逻辑	金融转账 / 合同审查

MCP 可以帮助 Agent 发现工具，A2A 可以帮助 Agent 建立通信，RSF 则帮助 Agent 更加明确地表达任务、状态、条件、目标和决策依据。三者可以共存。

2.6 三个核心特性

RSF 在为智能系统提供语义接口时，围绕三个核心特性构建其价值。

一致性（Consistency）：所有参与通信的智能体基于同一个语义结构工作。当 Agent-A 将信息提取为六槽语义包后，Agent-B 和 Agent-C 读取的是完全相同的槽位结构，不需要各自从自然语言中重新解析语义。这意味着同一个事件不会因为不同 Agent 的语言理解差异而产生不同的解读。一致性还体现在跨语言场景中——同一事件在英文和中文中的六槽骨架完全一致，不存在“语种切换导致语义走样”的问题。

可追溯性（Traceability）：每个输出片段都可以回映到原始输入中的具体语言片段，并定位到对应的 A-F 槽位。Agent 可以追溯到“当前信息来自原始输入的哪句话、对应哪个槽位”。六槽结构本身就是一个天然的索引体系：当系统需要回答“这个金额来自哪里”时，可以沿着 Agent 链回溯到最初提取该金额的具体槽位和时间点。可追溯性使得语义路径不再是黑盒，而是一条可复查的链。

可审计性（Auditability）：在可追溯的基础上，RSF 的审计模式可以提供完整的结构化日志——每个 Agent 的输入输出均为六槽 JSON，每一步的槽位值、调整记录、决策依据都可以被逐项检查。对于金融、法律、医疗等高可靠性场景，这意味着系统可以回答“为什么得出了这个结论”——不是通过检查不可解释的模型内部状态，而是通过检查结构化的语义路径。

这三个特性相互支撑：一致性确保所有 Agent 在同一语义基准上工作，可追溯性确保每个决策路径可以被回溯，可审计性确保整个推理链可以被外部验证。它们共同构成了 RSF 区别于纯自然语言通信或普通 JSON Schema 的核心差异。

3. 设计原则

3.1 固定骨架 (Fixed Skeleton)

RSF 最基础的设计决策是：顶层槽位固定为六个，不得再增。

这个决策的出发点不是“六个维度足以描述一切语义”——没有任何有限结构能做到这一点。出发点是**闭包 (Closure)**：在六槽的边界内，任何事件型的语义表达都能找到归属。超出六槽边界的信息交由 AI 自身的智能处理。

固定骨架的价值在于：AI 系统不需要在每次推理时重新判断“这句话的结构应该是什么”——答案永远是那六个槽位。

RSF 假设底层 AI 系统已具备基本的自然语言理解能力，能够识别槽位边界并执行递归解析——这一假设在当前主流 LLM 上已可验证。

3.2 打包优先 (Packaging First)

传统语言分析倾向于**拆分**——把动词拆成时态、情态、语态等，把固定搭配拆成独立单词。拆分越细，语义片段之间的关系越复杂，组合的代价越高。

RSF 反其道而行之：**整体打包**。

B 槽整体化：一个动作的完整语义——动词本身及其时态、情态、固定搭配等——整体归入 B 槽。例如 “should have been completed” 整体入 B，而不是拆成四个独立成分。

F 槽统一化：方式、工具、原因等所有修饰性语义统一归入 F 槽，Core 层面不做细分。

打包优先的直接收益是：AI 不需要在推理时重新组装这些成分之间的语法关系。时态、情态等已经被锁定在 B 槽内部，AI 只需整体理解。

3.3 可追溯优先 (Traceability First)

RSF 的核心优先级是：**保真和可追溯优先于可读性**。每个输出片段必须能够回映到 A-F 槽位，关键信息（事实、情态、否定、数量、人称、因果强度）必须完整保留。当可读性与保真发生冲突时，RSF 宁可牺牲一定的可读性，也不允许信息丢失或槽位模糊。

在保证保真和可追溯的前提下，再优化输出的自然度。

3.4 扁平与递归

RSF 同时坚持两个看似矛盾的设计：结构扁平与递归深度。实际上两者并不矛盾。

扁平指的是：顶层只有六个槽位，不设子类槽、不引入 Topic/Focus 等必选标记层。这是一个宽度上的约束。

递归指的是：当任何一个槽位内部包含一个完整事件时，可以用同一套 A - F 进入下一层，不增设新槽名。这是一个深度上的能力。

递归不违反扁平，因为递归不增加新的槽位类型——无论深入多少层，使用的仍然是 A、B、C、D、E、F。

3.5 回译守恒

RSF 定义了一个严格的检验标准——回译守恒：将文本从源语言转换为六槽结构，再从六槽结构生成目标语言，然后将目标语言翻译回源语言——必须与原始输入在关键信息上一致。情态、时体、否定、数量、人称、因果强度，任何一个维度的偏差都意味着保真度失败。

3.6 Core 的闭包

Core 追求闭包（Closure），而非完整（Completeness）。

4. 应用示例

4.1 简单事件：便利店购买

从第一章的便利店故事开始。游客购买两瓶可乐这一事件，用 RSF 表示为：

A	B	C	D	E	F
Customer	purchase	two bottles of cola	∅	convenience store	payment=\$2; condition=no common language

第一章的便利店故事中已经包含了完整的六槽语义，只是当时没有标注出来。RSF 所做的，正是将这些隐含的语义角色显式化。

4.2 翻译保真示例

The report should have been completed by the team last week.

A	B	C	D	E	F
---	---	---	---	---	---

the team	should have been completed	the report	last week	∅	∅
----------	----------------------------	------------	-----------	---	---

传统语法拆分会将 “should have been completed” 拆解为情态动词 (should) + 完成体 (have been) + 被动语态 (completed)。RSF 将其整体打包入 B 槽，因为这三个成分共同表达了一个不可分割的语义单元：“过去应该已经被完成但不确定是否完成”。

回译守恒检查：从 B 槽 “should have been completed” 生成中文“应该已经被完成”，再回译回英文——时态、情态、被动语态全部保留，不丢失、不减弱。

4.3 多 Agent 链式通信：退货处理

这是最重要的案例，展示 RSF 如何在多个 Agent 之间传递语义，同时阻断语义漂移。

场景

一个电商系统的三个 Agent 协作处理客户退货：

Agent	角色
Agent-A	客服 Agent，接收客户请求
Agent-B	仓库 Agent，处理退货入库与质检
Agent-C	财务 Agent，执行退款

步骤 1：客户输入

“我要退掉上周买的那台有缺陷的 BrandX Pro-15 笔记本电脑，订单号是 ORD-20260703-8821。”

步骤 2：Agent-A 提取 RSF 语义包

```
{
  "A": "Customer (CUST-48291, 张三)",
  "B": "return + request_inspection + conditional_refund",
  "C": {
    "item": "BrandX Pro-15 Laptop",
    "order_id": "ORD-20260703-8821"
  },
  "D": {
    "purchase_date": "2026-07-03",
    "return_initiated": "2026-07-10"
  },
  "E": "∅",
  "F": {
    "defect_description": "intermittent_black_screen",
    "return_label": "issued_by_Agent-A_via_email",
    "condition_for_refund": "confirm_defect_first",
  }
}
```



```
    "unresolved_question": ["does_refund_include_original_shipping"]
  }
}
```

注意 F 槽中的 `unresolved_question` 字段：Agent-A 主动标注了自己不确定的信息（退款是否包含原始运费），而不是假装信息完整。

步骤 3: Agent-A → Agent-B（传递语义包）

Agent-A 将完整的 RSF 语义包传递给 Agent-B，不需要生成自然语言。

步骤 4: Agent-B 读取并处理

Agent-B 直接读取六槽结构：

- A = Customer CUST-48291 → 可查客户历史记录
- B = return + inspection + conditional_refund → 明确自己的任务链
- C = BrandX Pro-15, order ORD-20260703-8821 → 精确的 SKU 和订单号
- F.defect_description = intermittent_black_screen → 质检的具体标准
- F.unresolved_question = does_refund_include_original_shipping → Agent-B 知道这个问题尚未解决

Agent-B 执行质检后，将结果更新到语义包中，传递给 Agent-C：

```
{
  "A": "Agent-B (Warehouse-R2D2)",
  "B": "confirm_defect + forward_for_refund",
  "C": {
    "order_id": "ORD-20260703-8821",
    "customer": "CUST-48291",
    "item": "BrandX Pro-15"
  },
  "D": "2026-07-11",
  "E": "warehouse-R2D2",
  "F": {
    "inspection_result": "confirmed_intermittent_black_screen",
    "refund_authorization": {
      "approved": true,
      "amount": 8999.00,
      "currency": "CNY",
      "includes_shipping": true,
      "policy_reference": "refund_policy_v4.2_section_3.1"
    },
    "resolved_questions": {
      "does_refund_include_original_shipping": "yes_per_policy_v4.2"
    },
    "action_required_by": "Agent-C (Finance)"
  }
}
```

```
    }  
}
```

步骤 5: Agent-C 直接执行

Agent-C 收到后不需要回问任何问题，直接执行退款。

对比: NL 通信 vs RSF 语义包

维度	NL 通信	RSF 语义包
信息精确度	“有缺陷的笔记本电脑”——模糊	brand/model/order_id/defect 全部入槽，零歧义
确认轮次	至少 1-2 轮回问	0 轮额外确认
未解决问题	Agent-A 可能忘记提“运费是否包含”	F.unresolved_question 主动标记，问题不会丢失
语义漂移	经过两次 NL 编解码，信息可能衰减	直接传递，零漂移
审计	依赖 NLP 理解文本日志	逐槽检查结构化 JSON

Audit 轨迹

```
Input (Customer NL)  
↓  
Agent-A: RSF Extraction  
├─ A: Customer CUST-48291  
├─ B: return + inspection + conditional_refund  
├─ C: BrandX Pro-15 / ORD-20260703-8821  
├─ F: unresolved_question [shipping_fee]  
↓  
Agent-B: Inspection + Update  
├─ B: confirm_defect → forward_for_refund  
├─ F: resolved_questions [shipping_fee: yes]  
├─ F: refund_authorization [amount: 8999.00]  
↓  
Agent-C: Execute Refund  
├─ Action: refund_approved  
├─ Amount: 8999.00 CNY  
├─ Policy: refund_policy_v4.2_section_3.1
```

4.4 高保真领域翻译示例

RSF 的六槽结构和打包优先原则，在法律、金融、医学等对信息保真度要求极高的领域中具有天然适用性。以下两个示例展示 RSF 如何在这些领域中精确传递关键语义。

法律条款

“The Lessee shall, within 30 days after receipt of the notice, remedy the breach, failing which the Lessor may terminate this Agreement.”

A	B	C	D	F
Lessee	shall remedy	the breach	within 30 days after receipt of notice	condition=failing which → Lessor may terminate

保真点：B 槽的 “shall remedy” 保留了法律英语中 “shall” 的义务强度——它既不是 “may”（许可），也不是 “must”（命令性更强），而是法律义务的标准情态。F 槽捕获了 “failing which” 这个条件关系，这是法律条款中关键的逻辑连接——如果承租人未在30天内补救，则触发出租人的终止权。传统翻译可能将 “failing which” 简化为“否则”，丢失了法律文本中“条件→后果”的精确逻辑关系。

RSF 翻译：“承租人应在收到通知后30天内补救该违约行为，否则出租人可终止本协议。”

医学记录

“No evidence of metastasis was found in any of the 12 lymph nodes examined.”

A	B	C	F
∅	was found	no evidence of metastasis	scope=all 12 lymph nodes examined; negation=full

保真点：这是一个典型的医学报告句子，否定范围和量化信息是关键。C 槽 “no evidence of metastasis” 和 F 槽的 “scope=all 12 lymph nodes examined” 共同保留了否定的范围——不是“部分淋巴结未发现”，而是“所有 12 个淋巴结均未发现”。如果这里的否定范围被误读（例如理解为“12 个淋巴结中有部分未发现”），将直接影响临床诊断。A 槽留空也忠实地反映了原文未指明执行者的医学写作惯例。

RSF 翻译：“所检查的12枚淋巴结中均未发现转移证据。”

4.5 RAG 结构化示例

在 RAG 场景中，用户查询可以先被提取为六槽结构，再用于检索。例如查询“去年第三季度的营收数据”：

A	B	C	D	E	F
∅	query	营收数据	Q3 2025	∅	data_source=financial_reports

检索系统可以直接使用 D 槽（Q3 2025）和 C 槽（营收数据）的组合进行精确检索，而不是对整句 NL 做向量匹配。这可以提升结构化查询场景的检索精度。

5. 边界与定位

5.1 与相关工作的区别

AMR (Abstract Meaning Representation): AMR 将句子表示为有向图，节点是语义概念，边是关系。AMR 追求语言学完备性，但图的复杂度高、解析成本大。RSF 追求的是最小可执行事件骨架，而非完备语义图。

SRL (Semantic Role Labeling): SRL 识别句子中的谓词-论元结构，角色集随谓词类型变化。RSF 是固定六槽，固定意味着更低的解析复杂度，也意味着更粗的粒度——这是为工程可执行性所做的取舍。

FrameNet: FrameNet 基于框架语义学，每个词激活一个框架。FrameNet 极其丰富，但覆盖范围有限。RSF 不采用框架体系。

CNL (Controlled Natural Language): CNL 通过限制语法和词汇来消除歧义。RSF 不是 CNL——它不限制用户的输入方式，接受完整的自然语言输入，在理解层提取语义结构。

MCP / A2A: 通信协议管“怎么传”，RSF 管“传什么”。可以共存。

5.2 RSF 的定位

RSF 优先保证“最小可执行事件骨架 + 工程可审计性”，而非语言学完备性。

6. 当前状态、局限性与未来方向

6.1 当前状态

RSF 目前处于概念框架与初步案例验证阶段。核心规范（宪法）已冻结，但尚无自动化 RSF 解析器或大规模系统部署。本文档中的所有示例均为人工标注或基于 LLM 辅助提取。

6.2 已知局限性

尚无自动化解析器: 当前六槽提取由人工完成或依赖 LLM 辅助，尚未有专门的 RSF 解析引擎。从自然语言到六槽的自动化转换是实现 RSF 的关键瓶颈。

覆盖范围未系统验证: 六槽能否覆盖所有语言的所有表达类型？目前有初步人工验证，但无大规模测试。边界案例（如诗歌、隐喻、反讽）的覆盖程度未知。

Agent 通信未在真实系统中部署: 第四章的 Agent 示例是概念演示，尚未在真实多 Agent 系统中实现并验证其收益。token 节省、轮次减少等量化指标有待实际测量。

与现有系统的集成成本未知：将一个已有 Agent 系统从 NL 通信切换到 RSF 语义包通信，需要多少改造工作？目前没有实践经验。

6.3 未来方向

Benchmark 设计：需要可复现的测试框架来评估 RSF 的实际收益。**对照实验设计：**同一任务分别使用 NL 通信和 RSF 语义包通信，测量 token 消耗、通信轮次、一致性、语义漂移幅度和审计通过率。

开源解析器：开发一个从自然语言到六槽结构的自动化解析器，可以是基于规则的系统、微调的 LLM 或混合方案。

跨语言验证：中文、英文、日文、阿拉伯文等差异显著的语言是否都能被六槽充分覆盖？边界在哪里？

编码器前端预处理：在 LLM 或 T5 等 Transformer 模型的编码器前端加入 RSF 预处理步骤，将长文本先提炼为六槽骨架再输入模型——这一方向目前在概念阶段，需要具体的工程实验来验证可行性。

声明

规范名：RSF（宪法版）

状态：Core 冻结

协议：CC BY 4.0

本文档为思想原创记录，欢迎非商业学术讨论与验证。

全文完